

Khulna University of Engineering & Technology
Department of Computer Science and Engineering

PROJECT REPORT

Code-Mix Sentiment Analysis

A Comparative Study of Classical, From-Scratch Neural, and
Pretrained Transformer Models on Bangla–English Code-Mixed Product Reviews

Md. Shifat Hasan (2107067)
Siam Basher (2107078)

Course: Natural Language Processing Laboratory (CSE 4122)
Year / Term: 4th Year, 1st Term
Lab Group: B1
Instructors: Dr. K. M. Azharul Hasan
Md. Nazirulhasan Shawon

September 23, 2026

Contents

1	Introduction	2
2	Related Work	3
3	Dataset	3
3.1	Source	3
3.2	Construction pipeline	3
3.3	Final corpus statistics	4
4	Methodology	4
4.1	Tokenization	4
4.2	Classical baselines	4
4.3	From-scratch neural models	4
4.4	Pretrained transformer	5
4.5	Evaluation protocol	5
4.6	Architecture and pipeline diagrams	5
5	Implementation	8
6	Results	8
7	Discussion	9
8	Limitations	10
9	Conclusion and Future Work	10

Abstract

Sentiment analysis of Bangla–English code-mixed text is complicated by the informal, script-mixing nature of real user-generated content, which standard sentiment tools are rarely built to handle. This report presents an end-to-end, code-driven pipeline that builds a 150,000-review, label-balanced corpus from the public BanglishRev e-commerce review dataset, automatically tags each review into one of four language conditions (English, Bangla, Banglish, and Bangla–English code-switched), and trains nine sentiment classifiers spanning three families: three classical bag-of-words baselines (N-Gram, Bag-of-Words, TF-IDF, each paired with logistic regression), five neural architectures implemented from scratch in PyTorch (ANN, RNN, LSTM, an additive-attention model, and a Transformer encoder), and a fine-tuned multilingual BERT. Models are validated with stratified 5-fold cross-validation and compared on a held-out test set of 15,000 reviews. TF-IDF with logistic regression achieves the highest test accuracy (70.1%), narrowly ahead of the from-scratch neural models; code-switched text is confirmed as the hardest language condition once macro-F1, rather than raw accuracy, is used to correct for its small, imbalanced test slice. The full pipeline, trained models, and an interactive Streamlit application for live inference and model comparison are included with this report.

1 Introduction

Bangla is one of the most widely spoken languages in South Asia, and a large share of its everyday digital text is not written in pure Bangla or pure English but in a mixture of both – *code-mixed* text, often using Romanized (Latin-script) Bangla, sometimes called “Banglish.” E-commerce product reviews are a particularly rich, naturally occurring source of this phenomenon: shoppers write freely, switching scripts and languages mid-sentence with no editorial constraint. This makes the domain both realistic and challenging for sentiment analysis, since most publicly available sentiment tools and pretrained resources are built around monolingual, formally-written text.

Objectives

This project set out to:

1. build a sentiment-labeled corpus of Bangla–English code-mixed product reviews entirely through code, with no hand-written or hand-translated examples;
2. automatically characterize each review’s language condition (English, Bangla, Banglish, or code-switched) so that model performance could be examined per condition, not just in aggregate;
3. implement and fairly compare three families of sentiment classifiers – classical statistical baselines, from-scratch neural networks, and a fine-tuned pretrained transformer – under a common, leakage-free cross-validation protocol; and
4. package the result as an interactive application so the comparison is directly explorable, not just reported as static numbers.

Contributions

The concrete contributions of this work are: (i) a fully automated, reproducible dataset-construction pipeline (Section 3); (ii) nine trained sentiment models spanning a wide capability range, from bag-of-words counting to a pretrained multilingual transformer (Section 4); (iii) a 5-fold cross-validated, held-out-test-evaluated comparison broken down by language condition (Section 6); and (iv) an interactive web application for live prediction, model comparison, and an accessible explanation of how each model works (Section 5).

2 Related Work

Classical text classification represents documents as sparse bag-of-words or TF-IDF-weighted vectors [2] and pairs them with a linear classifier; despite its simplicity, this remains a strong and well-understood baseline for sentiment tasks. Recurrent architectures such as the LSTM [3] were introduced to address the vanishing-memory problem of plain RNNs over long sequences, and additive (Bahdanau) attention [4] further let a recurrent model weight earlier time steps unequally rather than relying solely on a final hidden state. The Transformer [5] replaced recurrence entirely with self-attention, computing all pairwise token interactions in parallel, and its pretrained descendant BERT [6] – along with its multilingual variant, mBERT, trained jointly across 104 languages including Bangla – demonstrated that large-scale pretraining followed by task-specific fine-tuning outperforms training from scratch on most downstream NLP tasks, given enough fine-tuning data. Code-mixed sentiment analysis specifically has received growing attention as social media and e-commerce text increasingly reflects real multilingual usage rather than formal, monolingual writing; this project’s data source, BanglishRev [1], is one such recent large-scale resource for Bangla–English code-mixed e-commerce reviews.

3 Dataset

3.1 Source

The corpus is built entirely by code from **BanglishRev** [1], a public dataset of approximately 1.74 million reviews across roughly 128,000 products, released under CC-BY-NC-SA-4.0. Each raw record is a product with a nested list of reviews, a 1–5 star rating, a category label, and (unused in this project) associated review images.

3.2 Construction pipeline

The dataset is assembled by a fully automated pipeline with no manually written or translated examples:

1. **Flattening** the nested per-product JSON into one row per review, keeping only text and metadata columns (image fields are dropped by construction, never downloaded).
2. **Label derivation** from the star rating: 1–2 stars → *negative*, 3 stars → *neutral*, 4–5 stars → *positive*.
3. **Language-condition detection**, a Unicode-script and English-dictionary heuristic: a review is tagged *bangla* if Bangla-script tokens dominate above a 0.6 ratio threshold, *english* if Latin-script tokens dominate *and* most of them are real English-dictionary words, *banglish* if Latin-script tokens dominate but are mostly non-dictionary (Romanized Bangla), and *code_switched* otherwise.
4. **Cleaning and deduplication**: control characters stripped, near-empty or content-free reviews dropped (including garbled-encoding artifacts that would otherwise tokenize to nothing), exact duplicates removed.
5. **Balanced subsampling** to a target corpus size, balanced by label; language condition is left at its natural distribution within each label rather than forced equal, since code-switched reviews are intrinsically rare in this corpus.

Two data-quality issues were identified and corrected during development rather than left in the released corpus. First, an initial category filter intended to keep the corpus to “universally familiar” product types excluded roughly 74% of otherwise valid reviews (everyday items such as smartwatches, shampoo, or diapers did not match its narrow keyword list); the filter was broadened, since the source is already ordinary consumer e-commerce by construction. Second, a small number of reviews tokenized to zero content tokens (pure encoding artifacts, or stylized Unicode lettering used to evade filters); such reviews are excluded during cleaning, since an all-padding input was found to destabilize the attention-based

model during training (Section 4).

3.3 Final corpus statistics

The final corpus contains **150,000** reviews, exactly balanced by label. It is split into a 90% cross-validation pool (135,000 rows) and a 10% held-out test set (15,000 rows), stratified so both label and, where feasible, language condition are preserved in each split.

Table 1: Final corpus composition.

	Count	Share
<i>By label</i>		
Positive	50,000	33.3%
Negative	50,000	33.3%
Neutral	50,000	33.3%
<i>By language condition</i>		
Bangla	55,851	37.2%
English	50,555	33.7%
Banglish	41,905	27.9%
Code-switched	1,689	1.1%

Code-switched reviews are a genuine minority in this corpus – about 1.1% overall, and only 1,689 of the full 1.74 million raw reviews meet the detector’s code-switching criteria at all. This is treated as an honest property of the underlying data rather than corrected by duplication or synthetic generation, consistent with the project’s fully code-driven, non-fabricated data policy; its effect on evaluation robustness is discussed in Section 7.

4 Methodology

4.1 Tokenization

A custom tokenizer handles all four language conditions uniformly: Bangla script (Unicode range U+0980–U+09FF) and other-script letter runs (covering Latin text and stylized Unicode letters, which are otherwise dropped by narrower ASCII-only patterns) are each captured as tokens, alongside digit runs and individual punctuation symbols. For the from-scratch neural models, the vocabulary is built from tokens occurring at least 3 times in the training data, yielding **24,787** tokens; this threshold bounds the embedding table’s size and avoids fitting noise from one-off typos in a 100k+ document corpus.

4.2 Classical baselines

Three classical feature representations are each paired with logistic regression: **N-Gram** (unigram and bigram counts), **Bag-of-Words** (unigram counts only), and **TF-IDF** (unigram and bigram, weighted to down-weight ubiquitous terms and up-weight distinctive ones). Each vectorizer is capped at 20,000 features.

4.3 From-scratch neural models

Five neural architectures are implemented in PyTorch, trained with randomly initialized weights (no pretraining), and share a 100-dimensional Word2Vec embedding space trained on the project’s own

training data:

Table 2: From-scratch neural model architectures.

Model	Architecture
ANN	Mean-pooled word embeddings $\rightarrow$ two dense layers (64, 32) $\rightarrow$ 3-class output. Order-insensitive by construction; serves as the neural lower bound.
RNN	Embedding $\rightarrow$ single-layer RNN (64-dim hidden state, sequential) $\rightarrow$ final hidden state $\rightarrow$ linear.
LSTM	Embedding $\rightarrow$ single-layer <i>bidirectional</i> LSTM (64-dim hidden state per direction) $\rightarrow$ concatenated final states (128-dim) $\rightarrow$ linear.
Attention	Embedding $\rightarrow$ single-direction LSTM retaining every time step’s hidden state $\rightarrow$ additive (Bahdanau) attention over all time steps $\rightarrow$ weighted context vector $\rightarrow$ linear.
Transformer	Embedding + sinusoidal positional encoding $\rightarrow$ 2-layer Transformer encoder (4 attention heads, feed-forward dimension 128) $\rightarrow$ mean-pool over non-padding tokens $\rightarrow$ linear.

All five use dropout of 0.3, a batch size of 64, and 4 training epochs – deliberately modest hyperparameters chosen so that CPU-only training of 5 models $\times$ 5 folds stays practical (roughly one hour total), given that dataset size (108,000 training rows per fold) already provides substantially more gradient updates per epoch than the project’s earlier, much smaller development corpus.

4.4 Pretrained transformer

`bert-base-multilingual-cased` (mBERT, approximately 178M parameters), pretrained by Google on Wikipedia text across 104 languages including Bangla, is fine-tuned end-to-end with a new 3-class classification head. Unlike the other eight models, BERT is *not* cross-validated: full 5-fold CV was estimated to cost several additional hours of CPU time for limited reporting benefit, so it is fine-tuned once, on a bounded 5,000-row training subsample (versus the $\sim 108,000$ rows the other models train on per fold) for 2 epochs, and evaluated on the same held-out test set as every other model. This deliberate scope reduction is a direct threat to a fully fair comparison and is revisited in Section 7.

4.5 Evaluation protocol

Classical and neural models are evaluated with stratified 5-fold cross-validation on the 135,000-row training pool. For the neural models, the tokenizer, Word2Vec embeddings, and class-balancing weights are rebuilt from *only* each fold’s training partition, never its validation partition, to avoid vocabulary leakage. After cross-validation, each model type is refit once on the *entire* training pool to produce the deployed checkpoint, which is then evaluated once on the untouched 15,000-row test set for the final comparison (Section 6). Class-weighted cross-entropy loss is used for the neural models to counteract the corpus’s natural label skew observed during raw data collection before balancing. All experiments fix a global random seed (42) for reproducibility.

4.6 Architecture and pipeline diagrams

Figure 1 traces how a raw BanglishRev review turns into nine trained models and a served prediction. Figures 2–8 then give a stage-by-stage view of each model family’s own forward pass. These diagrams are drawn by hand for the report, but they mirror – stage for stage – the diagrams the project’s own

`app/diagrams.py` auto-generates on the Streamlit app's Model Architecture page (Section 5) directly from each model's `forward()` method and from `src/config.py`'s live hyperparameters, so the two never drift apart.

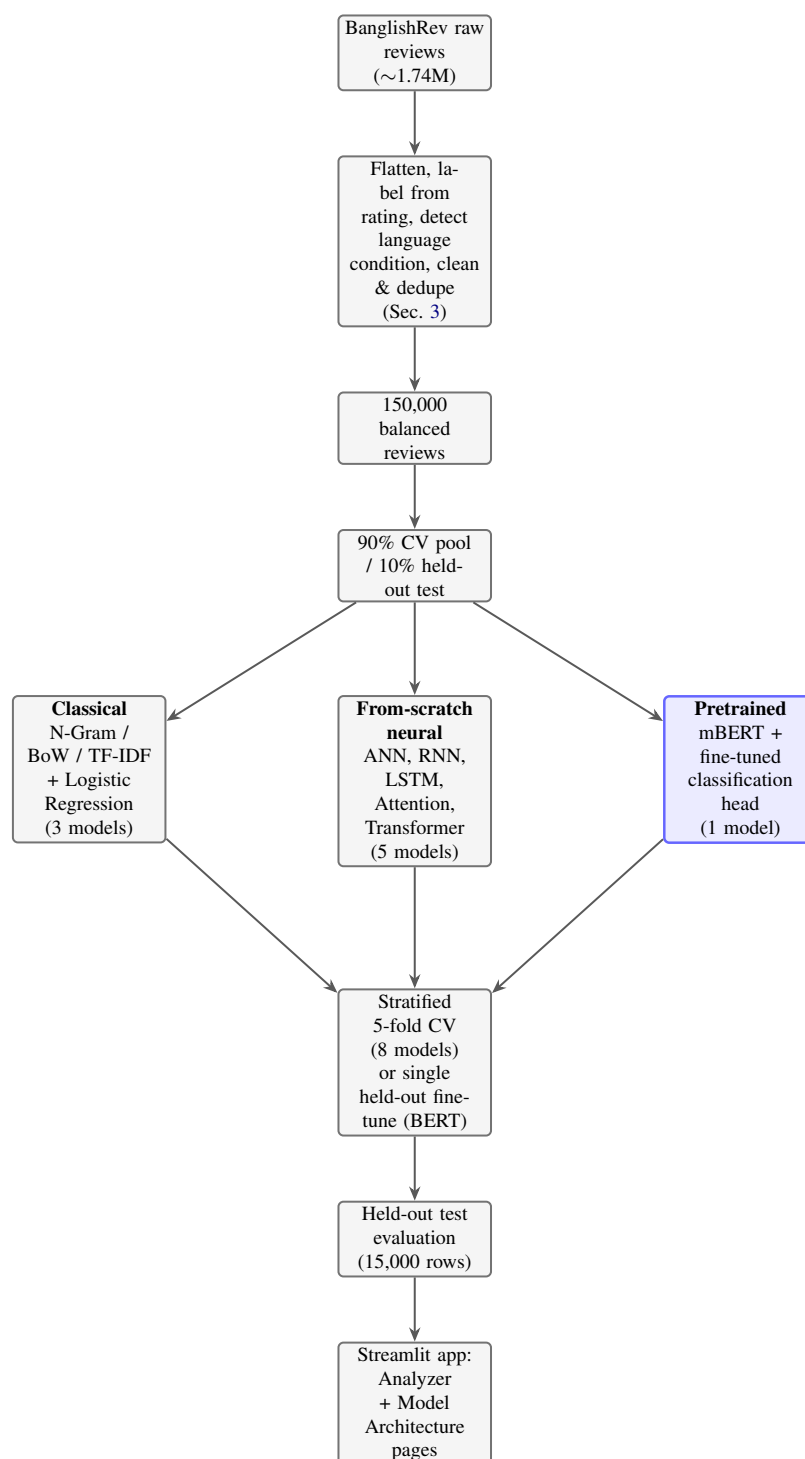


Figure 1: End-to-end project pipeline: dataset construction (Section 3) feeds three model families, which converge on a common evaluation protocol and the interactive application (Section 5).

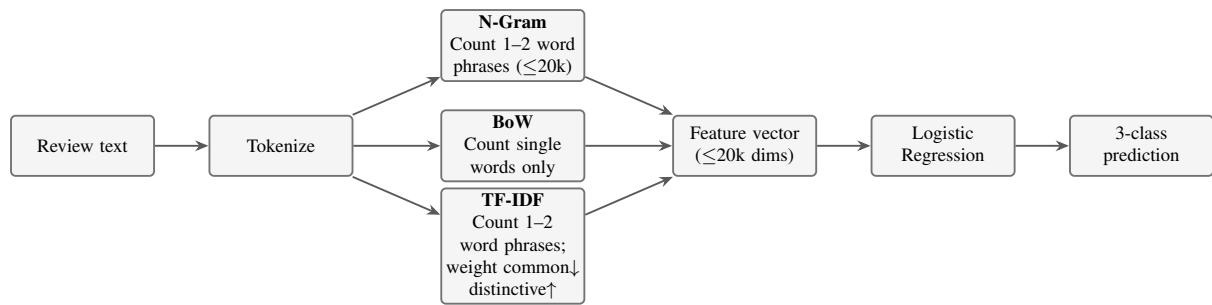


Figure 2: Classical baselines: all three feature representations share the same tokenize → count → vectorize → logistic-regression pipeline and differ only in how the counting/weighting stage works.

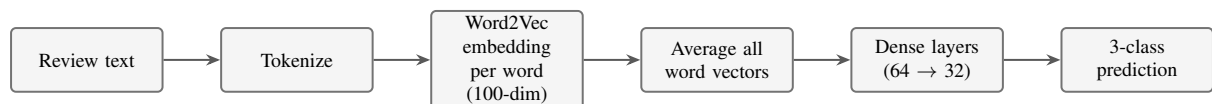


Figure 3: ANN: mean-pooled Word2Vec embeddings through two dense layers. Order-insensitive by construction – the project’s neural lower bound.

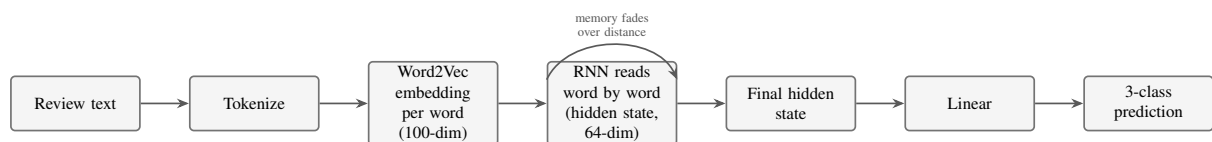


Figure 4: RNN: a single-layer recurrent encoder over word embeddings, classified from its final hidden state. The self-loop marks the vanishing-memory limitation LSTM is introduced to address.

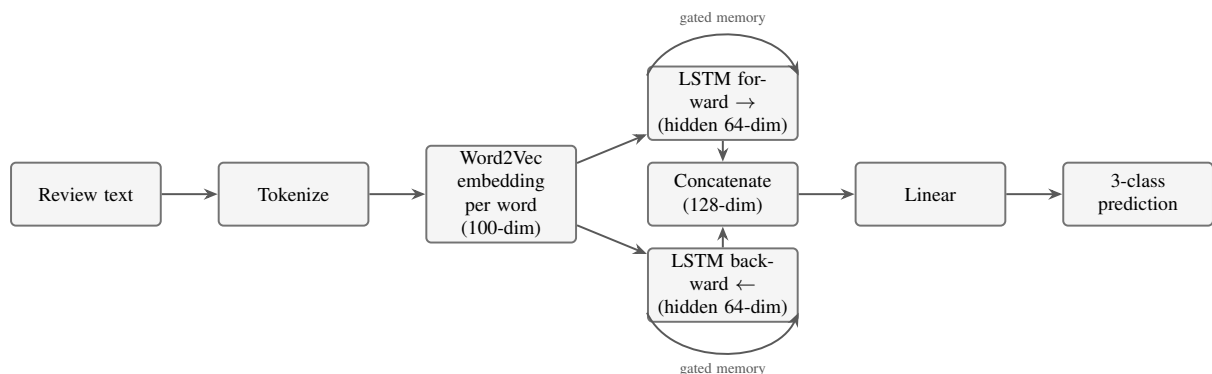


Figure 5: LSTM: a bidirectional, gated recurrent encoder; forward and backward final states are concatenated before classification.

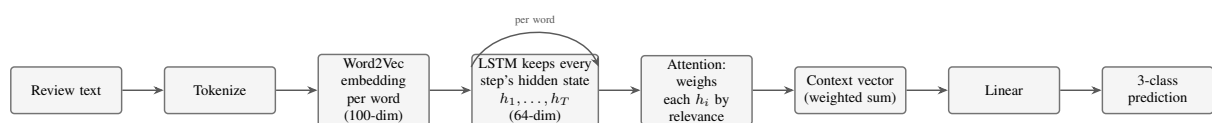


Figure 6: Additive (Bahdanau) attention: an LSTM encoder retains every time step’s hidden state, which attention then weighs and combines into a single context vector, instead of relying only on the final state.

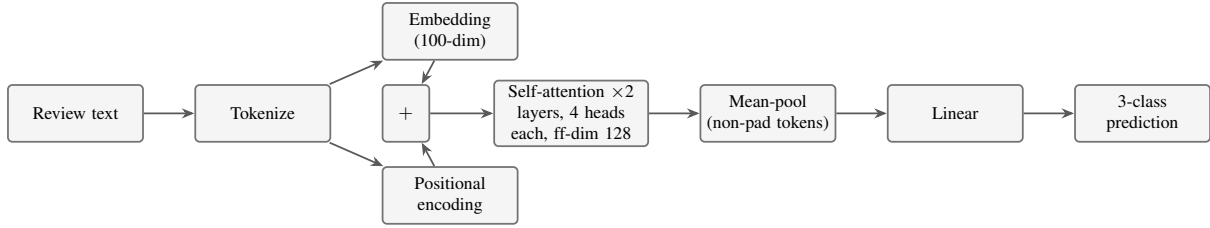


Figure 7: Transformer encoder: sinusoidal positional encoding is added to the token embedding before self-attention, since attention itself carries no notion of token order.

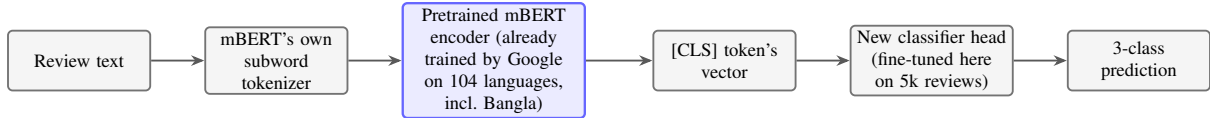


Figure 8: mBERT: the project’s only pretrained model, fine-tuned end-to-end with a new classification head on top of the [CLS] token’s contextual representation.

5 Implementation

The pipeline is implemented in Python with PyTorch (neural models), scikit-learn [7] (classical models and metrics), gensim (Word2Vec), and HuggingFace Transformers (mBERT). All paths, hyperparameters, and dataset constants are centralized in a single configuration module, so every script – dataset construction, training, evaluation – reads from one source of truth. An interactive **Streamlit** application is included, with two pages: an *Analyzer* page where a user types or samples a review and compares live predictions, confidence, and consensus across all nine models side by side; and a *Model Architecture* page that explains each model in plain language alongside an auto-generated pipeline diagram, with every quoted hyperparameter read live from the same configuration module used for training, so the documentation cannot drift out of sync with the code.

6 Results

Table 3: Cross-validation (5-fold, training pool) versus held-out test performance, sorted by test accuracy.

Model	CV accuracy (mean $\pm$ std)	Test accuracy	Test F1 (macro)
TF-IDF	70.25 $\pm$ 0.31%	70.11%	70.14%
ANN	68.43 $\pm$ 0.26%	68.74%	68.88%
Transformer	67.97 $\pm$ 0.30%	68.39%	68.56%
N-Gram	67.24 $\pm$ 0.27%	67.61%	67.57%
RNN	68.60 $\pm$ 0.32%	67.53%	67.64%
Bag-of-Words	67.55 $\pm$ 0.07%	67.50%	67.44%
LSTM	68.88 $\pm$ 0.38%	67.25%	67.27%
Attention	68.12 $\pm$ 0.33%	66.95%	66.97%
BERT [†]	—	65.46%	65.08%

[†] Single held-out fine-tune, not cross-validated; trained on a 5,000-row subsample versus $\sim 108,000$ rows for the other models (Section 4).

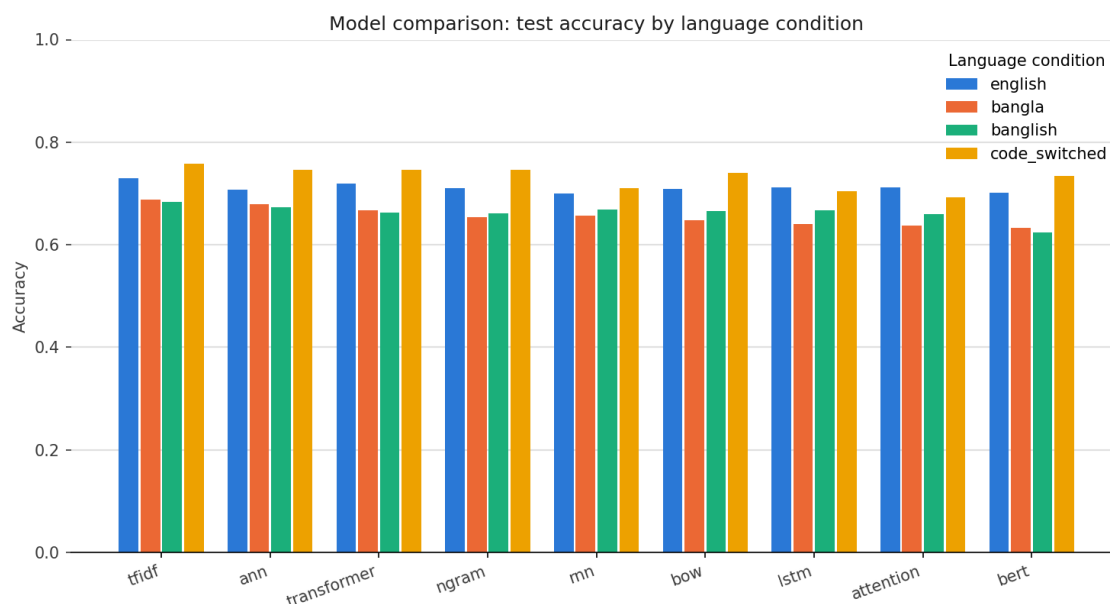


Figure 9: Test-set accuracy by model and language condition.

TF-IDF with logistic regression is the strongest model on both cross-validation and the held-out test set, and is the only model to clear 70% accuracy. The remaining eight models cluster within a relatively narrow 65–69% band; notably, the from-scratch neural models do not decisively outperform the simplest classical baselines at this corpus scale, and model rankings shift slightly between the cross-validation estimate and the single held-out test score (for example, LSTM ranks second by cross-validation mean but seventh on the test set) – a reminder that a single test-set number carries meaningful sampling variance even at 15,000 rows, and that the cross-validated mean is the more stable estimate of a model’s expected performance.

Breaking TF-IDF’s result down by language condition (Figure 9) shows accuracy of 72.9% on English, 68.7% on Bangla, 68.3% on Banglish, and an apparently strong 75.7% on code-switched text. However, the code-switched slice contains only 169 test examples, and its macro-F1 is just 59.8% – the *lowest* of the four conditions, versus 73.0% for English – revealing that the high raw accuracy is an artifact of class imbalance within that small slice rather than genuine robustness. This pattern held across every model examined, not only TF-IDF.

7 Discussion

Classical features remain competitive. At this corpus scale and with the modest neural hyperparameters used here, TF-IDF’s re-weighted bag-of-phrases representation was not surpassed by any neural architecture, including the Transformer. This does not imply neural methods are unsuited to the task – larger hidden dimensions, more training epochs, or embeddings pretrained on a larger external corpus could plausibly close or reverse the gap – but it is a useful, concrete reminder that architectural sophistication is not a substitute for adequate capacity and training budget, and that a strong classical baseline is worth measuring before assuming a neural model is necessary.

Code-switched robustness requires care in measurement. The project’s central motivating question – whether these models handle code-switched input as well as single-script input – cannot be answered by accuracy alone when a language condition is this underrepresented. Macro-F1 corrects for the label imbalance within the code-switched slice and shows it is, in fact, the hardest condition for every model, matching the intuitive expectation that intra-sentence language mixing is genuinely harder to model than

single-script text.

BERT’s result should not be read as “pretraining did not help.” BERT’s lower test accuracy is confounded by its much smaller fine-tuning set (5,000 rows versus $\sim 108,000$) and lower epoch budget, both deliberate concessions to CPU-only training time. A fairer comparison would fine-tune BERT on the same data volume as the other models, which this project’s compute budget did not accommodate.

8 Limitations

- **Heuristic language labels.** Language-condition tags are produced by an automated script-and-dictionary heuristic, not human annotation, and inherit that heuristic’s edge cases.
- **Code-switched scarcity.** With only 1.1% of the corpus and 169 test examples, conclusions about this condition carry wider uncertainty than the other three.
- **Unequal compute budgets.** BERT’s smaller fine-tuning sample and epoch count (Section 4) mean its result is not a fully controlled comparison against the other eight models.
- **Modest neural hyperparameters.** Hidden dimensions, epoch counts, and embedding dimensionality were deliberately kept small for CPU feasibility; larger budgets may change the ranking.

9 Conclusion and Future Work

This project delivered a fully code-driven, 150,000-review Bangla–English code-mixed sentiment corpus and a rigorously cross-validated comparison of nine sentiment models spanning classical, from-scratch neural, and pretrained transformer families, packaged with an interactive application for live exploration. TF-IDF with logistic regression was the strongest performer overall, and code-switched text was confirmed as the hardest language condition once evaluated with a class-imbalance-aware metric. Future work could fine-tune BERT on a training volume comparable to the other models, collect or synthesize additional code-switched examples under a clearly documented augmentation policy, and explore larger neural hyperparameter budgets now that the corpus scale supports them.

References

- [1] BanglishRev: A Large-Scale Bangla-English and Code-mixed Dataset of Product Reviews in E-Commerce. *arXiv preprint arXiv:2412.13161*, 2024.
- [2] G. Salton and C. Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.
- [3] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [4] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, 2019.
- [7] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.